Using Document Database (DocDB)

Version 2026.1
2026-04-20

InterSystems Corporation One Congress Street Boston MA 02114 www.intersystems.com

Using Document Database (DocDB)
PDF generated on 2026-04-20
InterSystems IRIS Version 2026.1

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)
Tel: +1-617-621-0700
Tel: +44 (0) 844 854 2917
Email: support@InterSystems.com

Table of Contents

1 Introducing InterSystems IRIS Document Database (DocDB) 1
1.1 Features and Benefits 1
1.2 Components of DocDB 2
1.2.1 Creating a Database 2
1.3 JSON Structure 3
1.3.1 Document 3
1.3.2 De-Normalized Data Structure 4
1.3.3 Data Types and Values 4
1.3.4 JSON Special Values 5

2 Managing Documents 7
2.1 Create or Get a Document Database 7
2.2 Define a Property: %CreateProperty() 8
2.3 Insert or Replace a Document: %SaveDocument() 8
2.4 Count Documents in a Database: %Size() 9
2.5 Get Document in a Database: %GetDocument() 9
2.6 Find Documents in a Database: %FindDocuments() 10
2.6.1 Restriction Predicate Array 10
2.6.2 Projection Predicate Array 11
2.6.3 Limit Predicate 11
2.7 Query Documents in a Database: %ExecuteQuery() 12
2.8 Delete a Document: %DeleteDocument() 13

3 REST Client Methods 15
3.1 Managing Databases 15
3.2 Managing Properties 16
3.3 Inserting and Updating Documents 16
3.4 Deleting Documents 17
3.5 Retrieving a Document 17

1

Introducing InterSystems IRIS Document Database (DocDB)

InterSystems IRIS® data platform DocDB is a facility for storing and retrieving database data. It is compatible with, but separate from, traditional SQL table and field (class and property) data storage and retrieval. It is based on JSON (JavaScript Object Notation) which provides support for web-based data exchange. InterSystems IRIS provides support for developing DocDB databases and applications in REST and in ObjectScript, as well as providing SQL support for creating or querying DocDB data.

By its nature, InterSystems IRIS Document Database is a schema-less data structure. That means that each document has its own structure, which may differ from other documents in the same database. This has several benefits when compared with SQL, which requires a pre-defined data structure.

The word “document” is used here as a specific industry-wide technical term, as a dynamic data storage structure. “Document”, as used in DocDB, should not be confused with a text document, or with documentation.

1.1 Features and Benefits

Some of the key features of InterSystems IRIS DocDB include:

- Application Flexibility: Documents do not require a predefined schema. This allows applications to rapidly set up their data environments and easily adapt to changes in data structure. This allows for rapid capture of data. Document Database can begin capturing data immediately, without having to define a structure for that data. This is ideal for unpredictable data feeds, as are often found in web-based and social media data sources. If in capturing a body of data, structures within that data become evident or emerge as useful, your document data structure can evolve. Existing captured data can co-exist with this more-structured data representation. It is up to your application to determine the data structure for each document and process it appropriately. One way to do this is to establish a key:value pair representing the document structure version. Thus, conversion of data from one JSON structure to another can be performed gradually, without interrupting data capture or access, or data conversion cannot be done at all.
- Sparse Data Efficiency: Documents are very efficient at storing sparse data because attributes with a particular key can appear in some documents in a collection but not in others. A document may have one set of defined keys; another document in the same collection may have a very different set of defined keys. In contrast, SQL requires that every record contain every key; in sparse data many records have keys with NULL values. For example, an SQL patient medical record provides fields for many diagnoses, conditions, and test; for most patients most of these fields are NULL. The system allocates space for all of these unused fields. In a DocDB patient medical record only those keys that contain actual data are present.

- Hierarchical Data Storage: DocDB is very efficient at storing hierarchically structured data. In a key:value pair data can be nested within the data to an unlimited number of levels. This means that hierarchical data can be stored de-normalized. In the SQL relational model, hierarchical data is stored normalized by using multiple tables.
- Dynamic Data Types: A key does not have a defined data type. The value assigned to the key has an associated data type. Therefore a key:value pair in one document may have one data type; a key:value pair for the same key in another document may have a different data type. Because data types are not fixed, you can change the data type of a key:value pair in a document at runtime by assigning a new value that has a different data type.

These features of DocDB have important implications for application development. In a traditional SQL environment, the database design establishes data structure that is followed in developing applications. In DocDB, data structure is largely provided in the applications themselves.

1.2 Components of DocDB

The package name for DocDB is %DocDB. It contains the following classes:

- %DocDB.Database: an ObjectScript persistent class used to manage the documents. A Database is a set of Documents, implemented by a persistent class that extends %DocDB.Document. You use methods of this class to create a database, retrieve an existing database, or delete a database, and within a database to insert a document, retrieve a document, or delete a document.
- %DocDB.Document: a structure used to store document data. It consists of the Document ID, the last modified date, and the document contents. The document content is stored in the %Doc property. Data is stored either as a JSON dynamic object or as a JSON dynamic array. A Document consists of multiple key:value pairs (an object) or an ordered list of values (an array).
- %DocDB.REST implements DocDB REST APIs to access the document database.

A related class is %Library.DynamicAbstractObject which is used to contain the JSON structures, and contains subclasses for JSON arrays and JSON key:value objects.

1.2.1 Creating a Database

A Database is an ObjectScript persistent class that extends the abstract class %DocDB.Document. You must instantiate a Database for each namespace used for DocDB. Only one database is required per namespace. Commonly, it is assigned the same name as the namespace name.

The following example shows how to create a Database through class definition:

```
Class MyDBs.People Extends %DocDB.Document [ DdlAllowed ]
```

The following example shows how to create a Database using the **%CreateDatabase()** method, specifying a package name:

ObjectScript

```
SET personDB = ##class(%DocDB.Database).%CreateDatabase("MyDBs.People")
```

The following example shows how to create a Database using the **%CreateDatabase()** method, taking the `ISC.DM` default package name:

ObjectScript

```
SET personDB = ##class(%DocDB.Database).%CreateDatabase("People")
```

The %SYSTEM.DocDB class provides an interface for managing Document Databases.

Refer to Managing Documents for a description of the API methods used to create or get a document database, to populate a database with documents, and to retrieve data from those documents.

1.3 JSON Structure

The InterSystems IRIS Document Database supports JSON dynamic objects and JSON dynamic arrays. You can create these JSON structures using the SET command.

The following example shows how hierarchical data can be stored using JSON. The first SET creates a dynamic abstract object containing nested JSON-structured key:value pairs and arrays. The example then converts the dynamic abstract object to a JSON string, then inserts that JSON string into an existing document database as a document.

ObjectScript

```
SET dynAbObj = {
 "FullName":"John Smith",
 "FirstName":"John",
 "Address":{
            "street":"101 Main Street",
            "city":"Mapleville",
            "state":"NY",
            "postal code":10234
           },
 "PhoneNumber":
            [
             {"type":"home","number":"212-456-9876"},
             {"type":"cell","number":"401-123-4567"},
             {"type":"work","number":"212-444-5000"}
            ]
}
SET jstring = dynAbObj.%ToJSON() // dynamic abstract object to JSON string
DO personDB.%FromJSON(jstring)   // JSON string inserted into document database
```

In this example, FullName is stored as a simple key:value pair. Address has a substructure which is stored as an object consisting of key:value pairs. PhoneNumber has a substructure which is stored as an array.

For further details refer to "Creating and Modifying Dynamic Entities" in Using JSON.

1.3.1 Document

A Document is stored in the *%Doc* property of an instance of the Database class you create. This is shown in the following example, which stores a JSON array in the %Doc property:

ObjectScript

```
SET jarry = ["Anne","Bradford","Charles","Deborah"]
SET myoref = ##class(MyDBs.DB1).%New()
SET myoref.%Doc = jarry
SET docoref = myoref.%Doc
WRITE "%Doc property oref: ",docoref,!
WRITE "%Doc Property value: ",docoref.%ToJSON()
```

By default, the %Doc data type is %Library.DynamicAbstractObject, which is the data type used to store a JSON object or a JSON array. You can specify a different data type in the **%CreateDatabase()** method.

Other Database properties:

- *%DocumentId* is an IDENTITY property containing a unique integer that identifies a document; %DocumentId counts from 1. In most cases, %DocumentId values are system-assigned. A %DocumentId must be unique; %DocumentIds

are not necessarily assigned sequential; gaps may occur in an assignment sequence. Document database also automatically generates an IdKey index for %DocumentId values.

- *%LastModified* records a UTC timestamp when the Document instance was defined.

1.3.2 De-Normalized Data Structure

The following is a JSON example of a traditional SQL normalized relational data structure. It consists of two documents, which might be contained in two different collections:

```
{
   "id":123,
   "Name":"John Smith",
   "DOB":"1990-11-23",
   "Address":555
}
{
   "id":555,
   "street":"101 Main Street",
   "city":"Mapleville",
   "state":"NY",
   "postal code":10234
 }
```

The following is the same data de-normalized, specified as a single document in a collection containing a nested data structure:

```
{
   "id":123,
   "Name":"John Smith",
   "DOB":"1990-11-23",
   "Address":{
              "street":"101 Main Street",
              "city":"Mapleville",
              "state":"NY",
              "postal code":10234
             }
 }
```

In SQL converting from the first data structure to the second would involve changing the table data definition then migrating the data.

In DocDB, because there is no fixed schema, these two data structures can co-exist as different representations of the same data. The application code must specify which data structure it will access. You can either migrate the data to the new data structure, or leave the data unchanged in the old data structure format, in which case DocDB migrates data each time it accesses it using the new data structure.

1.3.3 Data Types and Values

In DocDB, a key does not have a data type. However, a data value imported to DocDB may have an associated data type. Because the data type is associated with the specific value, replacing the value with another value may result in changing the data type of the key:value pair for that record.

InterSystems IRIS DocDB does not have any reserved words or any special naming conventions. In a key:value pair, any string can be used as a key; any string or number can be used as a value. The key name can be the same as the value: `"name":"name"`. A key name can be the same as its index name.

InterSystems IRIS DocDB represents data values as JSON values, as shown in the following table:

Data Value	Representation
Strings	String
Numbers	Numbers are represented in canonical form, with the following exception: JSON fractional numbers between 1 and -1 are represented with a leading zero integer (for example, 0.007); the corresponding InterSystems IRIS numbers are represented without the leading zero integer (for example, .007).
$DOUBLE numbers	Represented as IEEE double-precision (64–bit) floating point numbers.
Non-printing characters	JSON provides escape code representations of the following non-printing characters: $CHAR(8): "\b" $CHAR(9): "\t" $CHAR(10): "\n" $CHAR(12): "\f" $CHAR(13): "\r" All other non-printable characters are represented by an ecaped hexidecimal notation. For example, $CHAR(11) as "\u000b". Printable characters can also be represented using ecaped hexidecimal (Unicode) notation. For example, the Greek lowercase letter alpha can be represented as "\u03b1".
Other escaped characters	JSON escapes two printable characters, the double quote character and the backslash character: $CHAR(34): "\"" $CHAR(92): "\\"

1.3.4 JSON Special Values

JSON special values can only be used within JSON objects and JSON arrays. They are different from the corresponding ObjectScript special values. JSON special values are specified without quotation marks (the same values within quotation marks is an ordinary data value). They can be specified in any combination of uppercase and lowercase letters; they are stored as all lowercase letters.

- JSON represents the absence of a value by using the `null` special value. Because Document Database does not normally include a key:value pair unless there is an actual value, `null` is only used in special circumstances, such as a placeholder for an expected value. This use of `null` is shown in the following example:

  ObjectScript

  ```
  SET jsonobj = {"name":"Fred","spouse":null}
  WRITE jsonobj.%ToJSON()
  ```

- JSON represents a boolean value by using the `true` and `false` special values. This use of boolean values is shown in the following example:

ObjectScript

```
SET jsonobj = {"name":"Fred","married":false}
WRITE jsonobj.%ToJSON()
```

ObjectScript specifies boolean values using 0 and 1. (Actually “true” can be represented by 1 or by any non-zero number.) These values are not supported as boolean values within JSON documents.

In a few special cases, JSON uses parentheses to clarify syntax:

- If you define a local variable with the name null, true, or false, you must use parentheses within JSON to have it treated as a local variable rather than a JSON special value. This is shown in the following example:

  ObjectScript

  ```
  SET true=1
  SET jsonobj = {"bool":true,"notbool":(true)}
  WRITE jsonobj.%ToJSON()
  ```

- If you use the ObjectScript Follows operator (]) within an expression, you must use parentheses within JSON to have it treated as this operator, rather than as a JSON array terminator. In the following example, the expression `b]a` tests whether b follows a in the collation sequence, and returns an ObjectScript boolean value. The Follows expression must be enclosed in parentheses:

  ObjectScript

  ```
  SET a="a",b="b"
  SET jsonarray=[(b]a)]
  WRITE jsonarray.%ToJSON()
  ```

2

Managing Documents

InterSystems IRIS® data platform DocDB supplies class methods that enable you to work with DocDB from ObjectScript. For further details on invoking JSON methods from ObjectScript, see Using JSON.

2.1 Create or Get a Document Database

To create a new document database in the current namespace, invoke the **%CreateDatabase()** method.

To get an existing document database in the current namespace, invoke the **%GetDatabase()** method.

You assign a document database a unique name within the current namespace. The name can be qualified `"packagename.docdbname"` or unqualified. An unqualified database name defaults to the `ISC.DM` package.

The following example gets a database if a database with that name exists in the current namespace; otherwise, it creates the database:

ObjectScript

```
IF $SYSTEM.DocDB.Exists("People")
  { SET db = ##class(%DocDB.Database).%GetDatabase("People")}
ELSE {SET db = ##class(%DocDB.Database).%CreateDatabase("People") }
```

The following example creates or gets a database, then uses **%GetDatabaseDefinition()** to display the database definition information, which is stored as a JSON Dynamic Abstract Object:

ObjectScript

```
IF $SYSTEM.DocDB.Exists("People")
  { SET db = ##class(%DocDB.Database).%GetDatabase("People")}
ELSE {SET db = ##class(%DocDB.Database).%CreateDatabase("People") }
SET defn = db.%GetDatabaseDefinition()
WRITE defn.%ToJSON()
```

You can use **%SYSTEM.DocDB.GetAllDatabases()** to return a JSON array containing the names of all databases defined in this namespace.

2.2 Define a Property: %CreateProperty()

In order to retrieve a document by a key:value pair, you must use **%CreateProperty()** to define a property for that key. Defining a property automatically creates an index for that key which InterSystems IRIS maintains when documents are inserted, modified, and deleted. A property must specify a data type for the key. A property can be specified as accepting only unique values (1), or as non-unique (0); the default is non-unique.

The following example assigns two properties to the database. It then displays the database definition information:

ObjectScript

```
  IF $SYSTEM.DocDB.Exists("People")
    { SET db = ##class(%DocDB.Database).%GetDatabase("People")}
  ELSE {SET db = ##class(%DocDB.Database).%CreateDatabase("People") }
  DO db.%CreateProperty("firstName","%String","$.firstName",0) // creates non-unique property
  DO db.%CreateProperty("lastName","%String","$.lastName",1)   // create a unique property; an index
to support uniqueness is automatically created
  WRITE db.%GetDatabaseDefinition().%ToJSON()
```

2.3 Insert or Replace a Document: %SaveDocument()

You can insert or replace a document in a database using either the documentID or data selection criteria.

The **%SaveDocument()** and **%SaveDocumentByKey()** methods save a document, inserting a new document or replacing an existing document. **%SaveDocument()** specifies the document by documentId; **%SaveDocumentByKey()** specifies the document by key name and key value.

If you do not specify a documentId, **%SaveDocument()** inserts a new document and generates a new documentId. If you specify a documentId, it replaces an existing document with that documentId. If you specify a documentId and that document does not exist, it generates an ERROR #5809 exception.

The document data consists of one or more key:value pairs. If you specify a duplicate value for a key property that is defined as unique, it generates an ERROR #5808 exception.

The **%SaveDocument()** and **%SaveDocumentByKey()** methods return a reference to the instance of the database document class. This is always a subclass of %DocDB.Document. The method return data type is %DocDB.Document.

The following example inserts three new documents and assigns them documentIds. It then replaces the entire contents of the document identified by documentId 2 with the specified contents:

ObjectScript

```
  IF $SYSTEM.DocDB.Exists("People")
    { SET db = ##class(%DocDB.Database).%GetDatabase("People")}
  ELSE {SET db = ##class(%DocDB.Database).%CreateDatabase("People") }
    WRITE db.%Size(),!
  DO db.%CreateProperty("firstName","%String","$.firstName",0)
  SET val = db.%SaveDocument({"firstName":"Serena","lastName":"Williams"})
  SET val = db.%SaveDocument({"firstName":"Bill","lastName":"Faulkner"})
  SET val = db.%SaveDocument({"firstName":"Fred","lastName":"Astare"})
  WRITE "Contains ",db.%Size()," documents: ",db.%ToJSON()
  SET val = db.%SaveDocument({"firstName":"William","lastName":"Faulkner"},2)
  WRITE !,"Contains ",db.%Size()," documents: ",db.%ToJSON()
```

The following example chains the **%Id()** method to each **%SaveDocument()**, returning the documentId of each document as it is inserted or replaced:

ObjectScript

```
IF $SYSTEM.DocDB.Exists("People")
  { SET db = ##class(%DocDB.Database).%GetDatabase("People")}
ELSE {SET db = ##class(%DocDB.Database).%CreateDatabase("People") }
DO db.%CreateProperty("firstName","%String","$.firstName",0)
WRITE db.%SaveDocument({"firstName":"Serena","lastName":"Williams"}).%Id(),!
WRITE db.%SaveDocument({"firstName":"Bill","lastName":"Faulkner"}).%Id(),!
WRITE db.%SaveDocument({"firstName":"Fred","lastName":"Astare"}).%Id(),!
WRITE "Contains ",db.%Size()," documents: ",db.%ToJSON()
WRITE db.%SaveDocument({"firstName":"William","lastName":"Faulkner"},2).%Id(),!
WRITE !,"Contains ",db.%Size()," documents: ",db.%ToJSON()
```

2.4 Count Documents in a Database: %Size()

To count the number of documents in a database, invoke the **%Size()** method:

ObjectScript

```
SET doccount = db.%Size()
WRITE doccount
```

2.5 Get Document in a Database: %GetDocument()

To retrieve a single document from the database by %DocumentId, invoke the **%GetDocument()** method, as shown in the following example:

ObjectScript

```
DO db.%GetDocument(2).%ToJSON()
```

This method returns only the %Doc property contents. For example:

```
{"firstName":"Bill","lastName":"Faulkner"}
```

The method return type is %Library.DynamicAbstractObject.

If the specified %DocumentId does not exist, **%GetDocument()** generates an ERROR #5809 exception: “Object to load not found”.

You can retrieve a single document from the database by key value using **%GetDocumentByKey()**.

You can also return a single document from the database by %DocumentId, using the %FindDocuments() method. For example:

ObjectScript

```
DO db.%FindDocuments(["%DocumentId",2,"="]).%ToJSON()
```

This method returns the complete JSON document, including its wrapper:

```
{"sqlcode":100,"message":null,"content":[{"%Doc":"{\"firstName\":\"Bill\",\"lastName\":\"Faulkner\"}",
"%DocumentId":2,"%LastModified":"2018-03-06 18:59:02.559"}]}
```

2.6 Find Documents in a Database: %FindDocuments()

To find one or more documents in a database and return the document(s) as JSON, invoke the **%FindDocuments()** method. This method takes any combination of three optional positional predicates: a restriction array, a projection array, and a limit key:value pair.

The following examples, with no positional predicates, both return all of the data in all of the documents in the database:

ObjectScript

```
WRITE db.%FindDocuments().%ToJSON()
```

ObjectScript

```
WRITE db.%FindDocuments(,,).%ToJSON()
```

2.6.1 Restriction Predicate Array

The restriction predicate syntax `["property","value","operator"]` returns the entire contents of the matching documents. You specify as search criteria the property, value, and operator as an array. If you do not specify the operator, it defaults to `"="`. You can specify more than one restriction as an array of restriction predicates with implicit AND logic: `[["property","value","operator"],["property2","value2","operator2"]]`. The restriction predicate is optional.

The following example returns all documents with a documentId greater than 2:

ObjectScript

```
SET result = db.%FindDocuments(["%DocumentId",2,">"])
WRITE result.%ToJSON()
```

or by chaining methods:

ObjectScript

```
WRITE db.%FindDocuments(["%DocumentId",2,">"]).%ToJSON()
```

If the contents of documents match the search criteria, It returns results such as the following:

```
{"sqlcode":100,"message":null,"content":[{"%Doc":"{\"firstName\":\"Fred\",\"lastName\":\"Astare\"}","%DocumentId":"3","%LastModified":"2018-03-05
18:15:30.39"},{"%Doc":"{\"firstName\":\"Ginger\",\"lastName\":\"Rogers\"}","%DocumentId":"4","%LastModified":"2018-03-05
 18:15:30.39"}]}
```

If no documents match the search criteria, it returns:

```
{"sqlcode":100,"message":null,"content":[]}
```

To find a document by a key:value pair, you must have defined a document property for that key:

ObjectScript

```
IF $SYSTEM.DocDB.Exists("People")
  { SET db = ##class(%DocDB.Database).%GetDatabase("People")}
ELSE {SET db = ##class(%DocDB.Database).%CreateDatabase("People") }
  WRITE db.%Size(),!
DO db.%CreateProperty("firstName","%String","$.firstName",0)
SET val = db.%SaveDocument({"firstName":"Fred","lastName":"Rogers"})
SET val = db.%SaveDocument({"firstName":"Serena","lastName":"Williams"})
SET val = db.%SaveDocument({"firstName":"Bill","lastName":"Faulkner"})
SET val = db.%SaveDocument({"firstName":"Barak","lastName":"Obama"})
SET val = db.%SaveDocument({"firstName":"Fred","lastName":"Astare"})
SET val = db.%SaveDocument({"lastName":"Madonna"})
SET result = db.%FindDocuments(["firstName","Fred","="])
WRITE result.%ToJSON()
```

You can use a variety of predicate operators, including %STARTSWITH, IN, NULL, and NOT NULL, as shown in the following examples:

ObjectScript

```
SET result = db.%FindDocuments(["firstName","B","%STARTSWITH"])
WRITE result.%ToJSON()
```

ObjectScript

```
SET result = db.%FindDocuments(["firstName","Bill,Fred","IN"])
WRITE result.%ToJSON()
```

ObjectScript

```
SET result = db.%FindDocuments(["firstName","NULL","NULL"])
WRITE result.%ToJSON()
```

ObjectScript

```
SET result = db.%FindDocuments(["firstName","NULL","NOT NULL"])
WRITE result.%ToJSON()
```

2.6.2 Projection Predicate Array

To return only some of the values in returned documents you use a projection. The optional projection predicate, `["prop1","prop2",...]`, is an array listing the keys for which you wish to return the corresponding values. If you specify a user-defined key in the projection array, you must have defined a document property for that key.

The syntax `["property","value","operator"],["prop1","prop2",...]` returns the specified properties from the matching documents.

ObjectScript

```
SET result = db.%FindDocuments(["firstName","Bill","="],["%DocumentId","firstName"])
WRITE result.%ToJSON()
```

You can specify a projection with or without a restriction predicate. Thus both of the following are valid syntax:

- `db.%FindDocuments(["property","value","operator"],[prop1,prop2,...])` restriction and projection.
- `db.%FindDocuments(,[prop1,prop2,...])` no restriction, projection.

2.6.3 Limit Predicate

You can specify a limit key:value predicate, `{"limit":int}`, to return, at most, only the specified number of matching documents.

The syntax `["property","value","operator"],["prop1","prop2",...],{"limit":int}` returns the specified properties from the specified limit number of documents.

ObjectScript

```
SET result = db.%FindDocuments(["firstName","Bill","="],["%DocumentID","firstName"],{"limit":5})
WRITE result.%ToJSON()
```

This returns data from, at most, 5 documents.

You can specify a limit with or without a restriction predicate or a projection predicate. Thus all the following are valid syntax:

- `db.%FindDocuments(["property","value","operator"],,{"limit":int})` restriction, no projection, limit.
- `db.%FindDocuments(,[prop1,prop2,...],{"limit":int})` no restriction, projection, limit.
- `db.%FindDocuments(,,{"limit":int})` no restriction, no projection, limit.

The following example specifies no restriction, a projection, and a limit:

ObjectScript

```
WRITE db.%FindDocuments(,["%DocumentId","lastName"],{"limit":3}).%ToJSON()
```

2.7 Query Documents in a Database: %ExecuteQuery()

You can use the **%ExecuteQuery()** method to return document data from a database as a result set. You specify a standard SQL query SELECT, specifying the database name in the **FROM** clause. If the database name was created with no package name (schema), specify the default `ISC_DM` schema.

The following example retrieves the %Doc content data as a result set from all documents in the database:

ObjectScript

```
SET rval=db.%ExecuteQuery("SELECT %Doc FROM ISC_DM.People")
DO rval.%Display()
```

The following example uses a WHERE clause condition to limit what documents to retrieve by documentId value:

ObjectScript

```
SET rval=db.%ExecuteQuery("SELECT %DocumentId,%Doc FROM ISC_DM.People WHERE %DocumentId > 2")
DO rval.%Display()
```

The following example retrieve the %DocumentId and the lastName property from all documents that fulfill the **WHERE** clause condition. Note that to retrieve the values of a user-defined key, you must have defined a document property for that key:

ObjectScript

```
SET rval=db.%ExecuteQuery("SELECT %DocumentId,lastName FROM ISC_DM.People WHERE lastName %STARTSWITH
'S'")
DO rval.%Display()
```

For further information on handling a query result set, refer to Returning the Full Result Set or Returning Specific Values from the Result Set.

2.8 Delete a Document: %DeleteDocument()

You can delete documents from a database either by documentID or by data selection criteria.

- The **%DeleteDocument()** method deletes a single document identified by documentId:

  ObjectScript

  ```
  SET val = db.%DeleteDocument(2)
  WRITE "deleted document = ",val.%ToJSON()
  ```

  Successful completion returns the JSON value of the deleted document. Failure throws a StatusException error.

- The **%DeleteDocumentByKey()** method deletes a document identified by document contents. Specify a key:value pair.
- The **%Clear()** method deletes all documents in the database and returns the oref (object reference) of the database, as shown in the following example:

  ObjectScript

  ```
  SET dboref = db.%Clear()
  WRITE "database oref: ",dboref,!
  WRITE "number of documents:",db.%Size()
  ```

  This permits you to chain methods, as shown in the following example:

  ObjectScript

  ```
  WRITE db.%Clear().%SaveDocument({"firstName":"Venus","lastName":"Williams"}).%Id(),!
  ```

3

REST Client Methods

InterSystems IRIS® data platform DocDB REST Client API supplies methods that enable you to work with DocDB from REST. The REST API differs from other DocDB APIs because REST acts on resources while the other APIs act on objects. This resource orientation is fundamental to the nature of REST.

The curl examples in this topic specify port number 57774. This is only an example. You should use the port number appropriate for your InterSystems IRIS instance.

In curl the GET command is the default; therefore, a curl command that omits -X GET defaults to -X GET.

For DocDB the only valid Content-Type is application/json. If an unexpected Content-Type is requested then an HTTP Response Code = 406 is issued.

To use the REST API, you must enable the *%Service_DocDB* service. To do so, navigate to System Administration > Security > Services, select *%Service_DocDB*, and select the *%Service_DocDB* checkbox.

For further details on the REST API see %Api.DocDB.v1. For further details on REST, refer to Creating REST Services.

3.1 Managing Databases

- GetAllDatabases: Returns a JSON array which contains the names of all of the databases in the namespace.

```
curl -i -X GET -H "Content-Type: application/json" http://localhost:57774/api/docdb/v1/namespaceName
```

- DropAllDatabases: Deletes all of the databases in the namespace for which the current user has Write privileges.

```
curl -i -X DELETE -H "Content-Type: application/json"
http://localhost:57774/api/docdb/v1/namespaceName
```

- CreateDatabase: Creates a database in the namespace.

```
curl -i -X POST -H "Content-Type: application/json"
http://localhost:57774/api/docdb/v1/namespaceName/db/databaseName ?type= documentType& resource=
databaseResource
```

- GetDatabase: Returns the database definition of a database in the namespace.

```
curl -i -X GET -H "Content-Type: application/json"
http://localhost:57774/api/docdb/v1/namespaceName/db/databaseName
```

- DropDatabase: Drops a database from the namespace.

```
curl -i -X DELETE -H "Content-Type: application/json"
http://localhost:57774/api/docdb/v1/namespaceName/db/databaseName
```

3.2 Managing Properties

- CreateProperty: Creates a new property or replaces an existing property in the specified database. The property is defined by URL parameters and not Content. All parameters are optional.

```
curl -i -X POST -H "Content-Type: application/json"
http://localhost:57774/api/docdb/v1/namespaceName/prop/databaseName/
propertyName?type= propertyType& path= propertyPath& unique=propertyUnique
```

  The following example creates the City property in the Wx database:

```
http://localhost:57774/api/docdb/v1/mysamples/prop/wx/city?type=%String&path=query.results.channel.location.city&0
```

- GetProperty: Returns a property definition from the specified database.

```
curl -i -X GET -H "Content-Type: application/json"
http://localhost:57774/api/docdb/v1/namespaceName/prop/databaseName/propertyName
```

  The returned property definition is a JSON structure such as the following:

```
{"content":{"Name":"city","Type":"%Library.String"}}
```

- DropProperty: Deletes a property definition from the specified database.

```
curl -i -X DELETE -H "Content-Type: application/json"
http://localhost:57774/api/docdb/v1/namespaceName/prop/databaseName/propertyName
```

The following is a JSON property definition:

```
{"content":{"Name":"mydocdb","Class":"DB.MyDocDb","properties":[Array[5]
   0: {"Name":%%OTD,"Type":"%RawString"},
   1: {"Name":%Concurrency,"Type":"%RawString"},
   2: {"Name":%Doc,"Type":"%Library.DynamicAbstractObject"},
   3: {"Name":%DocumentId,"Type":"%Library.Integer"},
   4: {"Name":%LastModified,"Type":"%Library.UTC"}
   ]}}
```

3.3 Inserting and Updating Documents

- SaveDocument: Insert a new document into the specified database.

```
curl -i -X POST -H "Content-Type: application/json"
http://localhost:57774/api/docdb/v1/namespaceName/doc/databaseName/
```

  Note there is a slash at the end of this URI.

  To insert one or more documents, perform a POST. The body of the request is either a JSON document object or a JSON array of document objects. Note that the document objects may be in the unwrapped form with just content. This unwrapped form always results in an insert with a new %DocumentId. Specifying a wrapped document object whose %DocumentId is not present in the database results in an insert with that %DocumentId. Otherwise, the

%DocumentId property is ignored and an insert with a new %DocumentId takes place. If the request is successful, a single JSON document header object or an array of JSON document header objects is returned. If the request fails, the JSON header object is replaced by an error object.

- SaveDocument: Replace an existing document in the specified database.

```
curl -i -X PUT -H "Content-Type: application/json"
http://localhost:57774/api/docdb/v1/namespaceName/doc/databaseName/id
```

To insert a single JSON document object at a specific id location. If the specified %DocumentId already exists, the system replaces the existing document with the new document.

- SaveDocumentByKey: Replace an existing document in the specified database.

```
curl -i -X PUT -H "Content-Type: application/json"
http://localhost:57774/api/docdb/v1/namespaceName/doc/databaseName/keyPropertyName/keyValue
```

3.4 Deleting Documents

- DeleteDocument: Delete a document from the specified database.

```
curl -i -X DELETE -H "Content-Type: application/json"
http://localhost:57774/api/docdb/v1/namespaceName/doc/databaseName/id
```

Deletes the document specified by %DocumentId. If the request is successful, the specified document is deleted, the document wrapper metadata `{"%DocumentId":<IDnum>,"%LastModified":<timestamp>}` is returned, and a 200 (OK) status.

- DeleteDocumentByKey: Delete a document from the specified database.

```
curl -i -X DELETE -H "Content-Type: application/json"
http://localhost:57774/api/docdb/v1/namespaceName/doc/databaseName/keyPropertyName/keyValue
```

3.5 Retrieving a Document

- GetDocument: Return the specified document from the database.

```
curl -i -X GET -H "Content-Type: application/json"
http://localhost:57774/api/docdb/v1/namespaceName/doc/databaseName/id? wrapped=true|false
```

- GetDocumentByKey: Return a document by a property defined as a unique key from the database.

```
curl -i -X POST -H "Content-Type: application/json"
http://localhost:57774/api/docdb/v1/namespaceName/doc/databaseName/keyPropertyName/keyValue
```

FindDocuments: Return all documents from the database that match the query specification.

```
curl -i -X POST -H "Content-Type: application/json"
http://localhost:57774/api/docdb/v1/namespaceName/find/databaseName? wrapped=true|false
```

The following curl script example supplies full user credentials and header information. It returns all of the documents in the Continents document database in the MySamples namespace:

```
curl --user _SYSTEM:SYS -w "\n\n%{http_code}\n" -X POST
-H "Accept: application/json" -H "Content-Type: application/json"
http://localhost:57774/api/docdb/v1/mysamples/find/continents
```

It returns JSON data from the Document database such as the following:

```
{"content":{"sqlcode":100,"message":null,"content":[
  {"%Doc":"{\"code\":\"NA\",\"name\":\"North America\"}","%DocumentId":"1","%LastModified":"2018-02-15
 21:33:03.64"},
  {"%Doc":"{\"code\":\"SA\",\"name\":\"South America\"}","%DocumentId":"2","%LastModified":"2018-02-15
 21:33:03.64"},
  {"%Doc":"{\"code\":\"AF\",\"name\":\"Africa\"}","%DocumentId":"3","%LastModified":"2018-02-15
21:33:03.64"},
  {"%Doc":"{\"code\":\"AS\",\"name\":\"Asia\"}","%DocumentId":"4","%LastModified":"2018-02-15
21:33:03.64"},
  {"%Doc":"{\"code\":\"EU\",\"name\":\"Europe\"}","%DocumentId":"5","%LastModified":"2018-02-15
21:33:03.64"},
  {"%Doc":"{\"code\":\"OC\",\"name\":\"Oceana\"}","%DocumentId":"6","%LastModified":"2018-02-15
21:33:03.64"},
  {"%Doc":"{\"code\":\"AN\",\"name\":\"Antarctica\"}","%DocumentId":"7","%LastModified":"2018-02-15
21:33:03.64"}]}}
```

Restriction: The following curl script example restricts the documents returned from the Continents document database by specifying a restriction object. This restriction limits the documents returned to those whose name begins with the letter A:

```
curl --user _SYSTEM:SYS -w "\n\n%{http_code}\n" -X POST
-H "Accept: application/json" -H "Content-Type: application/json"
http://localhost:57774/api/docdb/v1/mysamples/find/continents -d
'{"restriction":["Name","A","%STARTSWITH"]}'
```

It returns the following JSON documents from the Document database:

```
{"content":{"sqlcode":100,"message":null,"content":[
  {"%Doc":"{\"code\":\"AF\",\"name\":\"Africa\"}","%DocumentId":"3","%LastModified":"2018-02-15
21:33:03.64"},
  {"%Doc":"{\"code\":\"AS\",\"name\":\"Asia\"}","%DocumentId":"4","%LastModified":"2018-02-15
21:33:03.64"},
  {"%Doc":"{\"code\":\"AN\",\"name\":\"Antarctica\"}","%DocumentId":"7","%LastModified":"2018-02-15
21:33:03.64"}]}}
```

Projection: The following curl script example projects which JSON properties to return from each document in the Continents document database. It uses the same restriction as the previous example:

```
curl --user _SYSTEM:SYS -w "\n\n%{http_code}\n" -X POST
-H "Accept: application/json" -H "Content-Type: application/json"
http://localhost:57774/api/docdb/v1/mysamples/find/continents -d
'{"restriction":["Name","A","%STARTSWITH"],"projection":["%DocumentId","name"]}'
```

It returns JSON data from the Document database such as the following:

```
{"content":{"sqlcode":100,"message":null,"content":[
  {"%Doc":"{"%DocumentId":"3","name":"Africa"}},
  {"%Doc":"{"%DocumentId":"4","name":"Asia"}},
  {"%Doc":"{"%DocumentId":"7","name":"Antarctica"}}]}}
```